

# 1. Python and Jupyter

## Functions, Debugging, and Profiling

DAEN 300  
**Alexander Abuabara**  
Fall 2026



# Learning Objectives

By the end of this presentation, you should be able to:

- ▶ Define and use Python functions.
- ▶ Use Jupyter/IPython tools to investigate errors.
- ▶ Read and interpret Python tracebacks.
- ▶ Measure execution time using `%time` and `%timeit`.
- ▶ Use profiling tools to identify performance bottlenecks.

# Why Jupyter and IPython?

Jupyter provides an interactive environment for working with Python.

- ▶ Execute code one cell at a time.
- ▶ Immediately inspect results.
- ▶ Combine code, text, equations, and visualizations.
- ▶ Experiment without repeatedly running an entire program.
- ▶ Use special IPython “magic” commands.

Examples of magic commands:

`%run`

`%timeit`

`%prun`

`%xmode`

# Check and Install Python Packages

Make sure a package is available before using it in Python.

**In a terminal or command prompt:**

```
# Show all installed packages
```

```
pip list
```

```
# Check one package
```

```
pip show pandas
```

```
# Install pandas if needed
```

```
pip install pandas
```

**In Jupyter Notebook:**

```
%pip install pandas
```

If installation fails, check that Python and pip are using the same environment.

# Load pandas and Read a CSV File

**Goal:** Import pandas, load a CSV file, and preview the data.

```
# Load the pandas package
import pandas as pd

# Read a CSV file
df = pd.read_csv("data.csv")

# View the first five rows
df.head()
```

**If the file is inside a folder:**

```
df = pd.read_csv("data/myfile.csv")
```

Replace data.csv with your own file name.

# Python Functions

A function packages reusable code into a named block.

```
def square(x):  
    return x ** 2  
  
result = square(5)  
print(result)
```

## Output:

25

Functions help make programs:

- ▶ reusable,
- ▶ easier to understand,
- ▶ easier to test,
- ▶ easier to debug.

# Functions with Multiple Inputs

Functions can accept multiple arguments.

```
def average(a, b):  
    return (a + b) / 2  
  
print(average(10, 20))
```

**Output:**

```
15.0
```

A good function should generally perform one clearly defined task.

# Why Functions Help

## Without a function:

```
result1 = 5 ** 2  
result2 = 8 ** 2  
result3 = 12 ** 2
```

## With a function:

```
def square(x):  
    return x ** 2  
  
result1 = square(5)  
result2 = square(8)  
result3 = square(12)
```

The second version is easier to reuse and modify.

# Running Python Files with %run

IPython allows Python scripts to be executed directly.

```
%run my_script.py
```

If `my_script.py` defines functions or variables, they can then be available in the interactive IPython session.

This is useful when:

- ▶ developing functions in separate Python files,
- ▶ testing scripts interactively,
- ▶ combining scripts with notebook analysis.

# What Is Debugging?

Debugging is the process of finding and correcting errors in a program.

Common Python errors include:

- ▶ `SyntaxError`
- ▶ `NameError`
- ▶ `TypeError`
- ▶ `IndexError`
- ▶ `KeyError`
- ▶ `ZeroDivisionError`

Python error messages provide information about where and why the program failed.

## Example: A Bug

Consider the following function:

```
def divide(a, b):  
    return a / b
```

```
divide(10, 0)
```

Python produces a traceback ending with:

```
ZeroDivisionError: division by zero
```

The traceback tells us:

1. what type of error occurred,
2. where it occurred,
3. which function calls led to it.

# Reading a Traceback

A useful strategy is to read a traceback from the bottom upward.

1. Identify the exception type.
2. Read the error message.
3. Find the line where the error occurred.
4. Follow the function-call chain if necessary.
5. Inspect the values involved.

The final line often provides the most important clue.

# IPython Traceback Modes

IPython provides the `%xmode` magic command.

```
%xmode Plain  
%xmode Context  
%xmode Verbose
```

The modes control how much information appears when an exception occurs.

- ▶ **Plain:** minimal traceback.
- ▶ **Context:** includes surrounding context.
- ▶ **Verbose:** displays additional variable information.

# Interactive Debugging

IPython also supports interactive debugging.  
After an exception:

```
%debug
```

This allows you to inspect the program state near the point of failure.

Useful debugger commands can include:

- ▶ inspecting variable values,
- ▶ moving through the call stack,
- ▶ identifying which inputs caused the error.

# Debugger Commands

ipdb> is the debugger prompt.

At that prompt, you can use commands such as:

```
n      # next line
s      # step into function
c      # continue execution
p x    # print x
pp x   # pretty-print x
u      # move up one stack frame
d      # move down one stack frame
q      # quit debugger
```

# What Is Profiling?

A program may be correct but still be slow.

**Profiling** means measuring where a program spends its time or memory.

The goal is to answer questions such as:

- ▶ Which function is slow?
- ▶ Which line takes the most time?
- ▶ Which operation uses the most memory?
- ▶ Where should optimization effort be focused?

**Measure first; optimize second.**

# Timing with %time

The %time magic measures one execution.

```
%time sum(range(1000000))
```

It reports information such as:

- ▶ CPU time,
- ▶ system time,
- ▶ total execution time.

It is useful for obtaining a quick timing estimate.

# Timing with %timeit

For more reliable measurements, use:

```
%timeit sum(range(1000000))
```

%timeit runs the statement repeatedly.

Advantages:

- ▶ reduces the effect of random timing variation,
- ▶ automatically chooses an appropriate number of repetitions,
- ▶ makes it easier to compare different approaches.

# Line Magic vs. Cell Magic

A single percent sign applies to one line:

```
%timeit sum(range(1000))
```

Two percent signs apply to an entire cell:

```
%%timeit  
  
total = 0  
for i in range(1000):  
    total += i
```

# Function Profiling with %prun

Suppose we have:

```
def calculate():  
    total = 0  
    for i in range(100000):  
        total += i ** 2  
    return total
```

Profile it with:

```
%prun calculate()
```

%prun reports which function calls consume the most time.

# More Profiling Tools

The Python Data Science Handbook also introduces more detailed profiling tools.

`%lprun` Line-by-line execution profiling.

`%memit` Memory usage of a statement.

`%mprun` Line-by-line memory profiling.

These tools can help identify a specific line of code responsible for poor performance.

# Recommended Workflow

A practical Jupyter development workflow:

1. Write a small function.
2. Run and test the function.
3. Inspect any traceback.
4. Debug the problem.
5. Confirm that the function is correct.
6. Measure its performance.
7. Profile slow sections.
8. Optimize only the important bottlenecks.

# Takeaways

- ▶ Functions organize Python code into reusable components.
- ▶ Jupyter makes experimentation and testing interactive.
- ▶ Tracebacks provide useful debugging information.
- ▶ IPython provides debugging tools such as `%xmode` and `%debug`.
- ▶ `%time` and `%timeit` measure execution time.
- ▶ Profiling reveals where optimization will have the greatest impact.

# Reference

Jake VanderPlas, *Python Data Science Handbook*

<https://jakevdp.github.io/PythonDataScienceHandbook/01.00-ipython-beyond-normal-python.html>

# Assignments

On Canvas!